Address Validator (Email Validation)

Objective : Main aim is to write a basic python program using string and conditional statements to find if the email_ID is valid or invalid.

Problem Statement : EmailID should be correct for proper communication, invalid emailID leads to communication failure. To make sure if the emailID is valid or invalid we are using string and conditional statements in python.

Validation Rules Used : 1.Remove spaces before and after in emailID.

2. Check if @ and dot(.) is in emailID.
3. check if @ comes before dot(.).
4. Check if @ and dot(.) should not start before or after emailID.
5. Depending on the conditions are satisfied, print valid or invalid.

Python Logic Explanation :

- List of Email_ID are given in the input.
- In a for loop block we are writing all the conditions to execute the block of code.
- Using strip() removes leading and trailing spaces.
- Condition_1 will check if @ and dot(.) is present in emailID.
- If block executes is @ coming before dot(.) in the emailID.
- To make sure the first and last characters are not @ and dot(.) using not equal to operator.
- Conditional statement IF will check if all the conditions are valid or invalid emailID in the given input.

Python code :#Title : Email_Address_validator

```
#check if samil_Email_ID are valid or invalid
Sample_email_ID = ['sravanthi@gmail.com',
                '    @sravanthi@gmail.com',
                'krish.gmail.com',
                'krish@gmail.com',
                '.abc@gmail.com',
                '   xyz@gmail.com.',
                'xyz@gmail.com',
                'helloworld.com    ',
                'helloworld@gmail.com',
                'abc@gmail@com']

for email_ID in Sample_email_ID :
  email_ID = email_ID.strip() # strip removes spaces in leading and trailing

#check if @ and dot(.) is in email_id
  condition_1 = '@' in email_ID and '.' in email_ID

#check if @ comes before dot(.)
  if condition_1 :
     order_valid = email_ID.find('@') < email_ID.find('.')
  else :
     order_valid = False

#check if the index [0] and [-1] of email_ID should not be @ and dot(.)
  first_char = email_ID[0]
  last_char = email_ID[-1]

#check if 1st and last character should not be @ and dot(.)
  first_last_char = ((first_char != '@' and  first_char != '.') and
                  (last_char != '@' and last_char != '.'))

#print email_ID next line
  print('\nEmail Entered :',email_ID)

#if all conditions are satisfied print valid else invalid
  if condition_1 and order_valid and first_last_char:
     print('Result       : VALID EMAIL')
  else:
     print('Result       : Invalid Email')
```

Sample Inputs and Outputs :

```
Email Entered : sravanthi@gmail.com
Result        : VALID EMAIL

Email Entered : @sravanthi@gmail.com
Result        : Invalid Email

Email Entered : krish.gmail.com
Result        : Invalid Email

Email Entered : krish@gmail.com
Result        : VALID EMAIL

Email Entered : .abc@gmail.com
Result        : Invalid Email

Email Entered : xyz@gmail.com.
Result        : Invalid Email

Email Entered : xyz@gmail.com
Result        : VALID EMAIL

Email Entered : helloworld.com
Result        : Invalid Email

Email Entered : helloworld@gmail.com
Result        : VALID EMAIL

Email Entered : abc@gmail@com
Result        : Invalid Email
```

Conclusion : This project successfully demonstrates how Python string methods and conditional logic can be used to validate email addresses. The program efficiently identifies valid and invalid email formats while maintaining clean and readable code.